You are a professional software engineer with expertise across multiple domains... ... Please evaluate the two diagrams (Model 1, Model 2) based on the provided requirement and scoring criteria.

Criteria
CRITICAL INSTRUCTION: You MUST evaluate the responses against EACH AND EVERY criterion listed in the "Specific Instructions" one by one. DO NOT aggregate, summarize, or skip any criterion.
Specific Instructions:
{{ Faithfulness / General / Five-critiera }}

Evaluation Guidelines
General Rules
- Consider only UML class diagram semantics (e.g., classes, attributes, associations, multiplicities, generalisation, composition/aggregation).
- Ignore layout, styling, skinparams, comments, and notes ... ...
2. **PlantUML Comments**: Using single-quote comments (e.g., `'opposite bank`) at the... ...

Requirement
{{ requirement }}
Model 1
{{ Model1 }}
Model 2

Choices
Please choose from the following 3 options based on the scoring criteria:
A. Response 1 is better.
B. Response 2 is better.
C. Tie.

Output
```Scores:
Choice: A/B/C
Reason: ... ...```

**Faithfulness**
- Ensure that the design aligns with the given requirement strictly, achieving all the functionalities based on the requirements without making any hallucinations and additions. Ensure that the conceptual classes and their relationships accurately represent the essentials outlined in the requirement. This includes a detailed focus on the associations between classes, their cardinalities, and the types of relationships such as inheritance, aggregation, and composition. Clarity in class names and the optional inclusion of attributes are key for aligning with the project's vision.

**General**
- Cohesion and Decoupling: The design should aim for high cohesion within individual classes and low coupling between different classes. High cohesion ensures that each class is dedicated to a singular task or concept, enhancing clarity and functionality. Low coupling reduces dependencies among classes, facilitating easier maintenance and scalability.
- Complexity: Utilize metrics such as the total number of classes, the average number of methods per class, and the depth of the inheritance tree to evaluate complexity. It's important to discern between conceptual classes and attributes; not every noun should become a class. The complexity level should be appropriately balanced, aligning with the specific requirements detailed in the project's requirement.
- Practicability: A practical design should be readable and understandable, offering a clear and comprehensive representation of the software's structures, functionalities, and behaviors. This enhances ease in programming, testing, and maintenance. Modularity should be evident, with each component serving a distinct function, streamlining the development process. Interfaces need to be designed for simplicity, facilitating smooth interactions within the software and with external environments. The design must also support robust testing strategies, enabling thorough validation through unit and acceptance tests, ensuring the design's viability in real-world applications.

**Five-critiera**
- Appropriate conceptual classes aligned with the domain
  1– Very Poor: Classes are mostly incorrect or irrelevant and do not accurately represent the domain.
  2– Poor: Only a few classes are correct, but the majority of the domain concepts are missing or incorrect.
  3– Fair: Some classes are correctly identified, but noticeable misalignments are still present.
  4– Good: Most classes are correctly identified with only minor misalignments
  5– Excellent: All classes are correctly identified, fully aligned with the domain.
- Associations/Multiplicities between classes for domain reasoning... ...
- Model attributes with appropriate data types .. ...
- Consistencies of Terminology with given context ... ...
- Adherence of UML class diagram standards ... ...
```